



What is The Rails Way

-The Basics and Beyond-

Composed By: Hayato Ishida

For Japanese version, click [here](#)

Updated On: 14 November 2025

About Me

- Accounts
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- Occupation: Software Engineer
- Things I Am Into
 - Linguistics
 - Singing at Karaoke
 - Listening to Music
 - Watching Movies
 - Cat Feeding



Licences / Certifications

- English
 - TOEIC® Listening & Reading 915: Certified on December 2019
- Engineering
 - Information Security Management: Certified on November 2017
 - Applied Information Technology Engineer: Certified on June 2017
 - Fundamental Information Technology Engineer: Certified on November 2016
 - IT Passport: Certified on April 2016
- Others
 - Abacus 2nd Class: Certified on June 2002
 - Mental Arithmetic 3rd Class: Certified on February 2001

Skill Stack

- Languages
 - Japanese: Native Proficiency
 - English: Full professional Proficiency
- Development
 - **Backend: Ruby(Ruby on Rails), Python**
 - Frontend: TypeScript + React.js, TypeScript +Vue.js
 - **Database: MySQL, PostgreSQL, MongoDB**
 - **Architecture: Monolith, Modular Monolith**
 - Hosting: AWS ESK
 - Versioning: Git, GitHub
 - **CI/CD: GitHub Actions, ArgoCD**
 - Monitoring: Datadog, Sentry

Work Histories

Period	Genre	Position	Jobs
Aug 2021 - Present	Saas(Educational Service)	Software Engineer	• Backend Development • Frontend Development • CI/CD Maintenance • Weekly Release Manager • Troubleshooting • Documentation • Technical Debt Resolution • Tech Blog Entries • LT Speaker
Feb 2020 - Dec 2020	SaaS(Chatbot Platform)	Backend Engineer	• Backend Development • Documentation • Technical Debt Resolution • Inspection of Alternative API

Work Histories

Period	Genre	Position	Jobs
Jun 2018 - Jan 2020	Contract-basis Developer	Software Engineer	• Backend Development • Frontend Development • Troubleshooting • Quality Assurance • Tech Blog Entries
Apr 2016 - Jan 2018	System Engineering Service	System Engineer	• Corporate Account Admin • Windows Server Maintenance • Security Admin Assistant • Internationalisation / Localisation

International-Exchange Activities

- Activities in University
 - English Linguistics Seminar(Focusing on Mass Media English)
 - International-Exchange Clubs(The 2nd Year)
 - [International-Exchange Programmes conducted by Japan Cabinet Office](#)(2013 - 2016)
 - Japanese Linguistics Course(The Final Year)
- Overseas Life Experience
 - Working Holiday in Australia(April 2014 - March 2015)
 - Language School for 1 month in Sydney
 - Work for 6 Months in [Hamilton Island Resort](#)
 - Volunteering for 1 Month as Assistant Teacher of Japanese Language at [St Ives High School](#) in NSW
- Other Activities
 - Keep Everyday Journal in English (April 2014 - Present)
 - [Sunrise Toastmasters Club](#)(February 2017 - March 2018)
 - [Vital Japan](#)(January 2018 - July 2019, October 2022 - February 2023)
 - Self Learning and Training of English Language
 - Participation Ruby-Related Tech Conferences

Agenda

1. [What brought me this interest?](#)
2. [Philosophy of "The Rails Way" -MVC vs. Layered-](#)
3. [Key Responsibilities -MVC vs. Layered-](#)
4. [Key Components of The Extended Rails Way\(Layered\)](#)
5. [Pros & Cons](#)
6. [Pragmatic Adoption Strategy](#)
7. [Survey and Answers](#)
8. [Summary](#)
9. [References](#)

1. What brought me this interest?



1. What brought me this interest?

My Pains at Optimisation of Messy Ecosystem of Rails App

- Nested domain namespaces: Systems and Accounts
 - * Illustrated on [the next slide](#)
- Inconsistencies in implementation of logics and place
- Too redundant layers
 - → *CQRS, Forms, Validators, Decorators, Serializers* etc.
- Unmanaged routing segregation between backend and frontend
- Not modular but "omnibus" monolith

→ No idea "What The Rails Way Is"!!

1. What brought me this interest?

Modular Monolith vs. Omnibus Monolith

```
monorepo/  
├── backend-app-1/  
│   ├── app/  
│   │   ├── assets/  
│   │   ├── channels/  
│   │   ├── controllers/  
│   │   ├── helpers/  
│   │   ├── jobs/  
│   │   ├── mailers/  
│   │   ├── models/  
│   │   └── views/  
│   ├── bin/  
│   │   ├── rails  
│   │   ├── rake  
│   │   └── setup  
│   ├── config/  
│   │   ├── environments/  
│   │   ├── initializers/  
│   │   ├── locales/  
│   │   ├── application.rb  
│   │   ├── boot.rb  
│   │   ├── database.yml  
│   │   ├── routes.rb  
│   │   └── storage.yml  
├── frontend-app-1/  
│   ├── public/  
│   ├── src/  
│   │   ├── assets/  
│   │   ├── components/  
│   │   ├── pages/  
│   │   ├── hooks/  
│   │   ├── context/  
│   │   ├── services/  
│   │   ├── utils/  
│   │   ├── store/  
│   │   ├── routes/  
│   │   ├── layouts/  
│   │   ├── types/  
│   │   └── __tests__/  
│   ├── App.jsx  
│   ├── App.css  
│   ├── index.jsx  
│   └── index.css  
├── .env  
├── .gitignore  
├── package.json  
├── package-lock.json  
├── README.md  
├── vite.config.js  
├── backend-app-2/  
├── frontend-app-2/  
└── k8s-dev/
```

vs.

```
omnirepo/  
├── app/  
│   ├── javascript/  
│   │   └── teacher_app/  
│   ├── commands/  
│   ├── controllers/  
│   │   ├── app1/  
│   │   ├── app2/  
│   │   ├── account_type_1/  
│   │   ├── account_type_2/  
│   │   └── account_type_3/  
│   └── app3/  
├── decorators/  
├── errors/  
├── factories/  
├── forms/  
├── helpers/  
├── mailers/  
├── models/  
└── workers/
```


1. What brought me this interest?

A Rubyist published the post on [Kaigi on Rails 2025](#), which said...

"What on earth is **The Rails Way**?"



1. What brought me this interest?



1. What brought me this interest?

The Reasons to Agree

- No official definition of "The Rails Way"
 - → **So many Rubyists, so many "The Rails Ways"**
- **MVC** ecosystem and/or **CoC** philosophy of the framework?
 - → **Too ambiguous to be unanimous**
- Different from *Omakase*?
 - → **Even DHH does not describe it as "The Rails Way"**

→ **No more consensus on a Rails app architecture with "The Rails Way".**

1. What brought me this interest?

✗ Now is not the time to cry

✓ **Now's the time to find out why "The Rails Way"**

Live Forever - Oasis

2. Philosophy of "The Rails Way" -MVC vs. Layered-



2. Philosophy of "The Rails Way" -MVC vs. Layered-

Traditional MVC Architecture

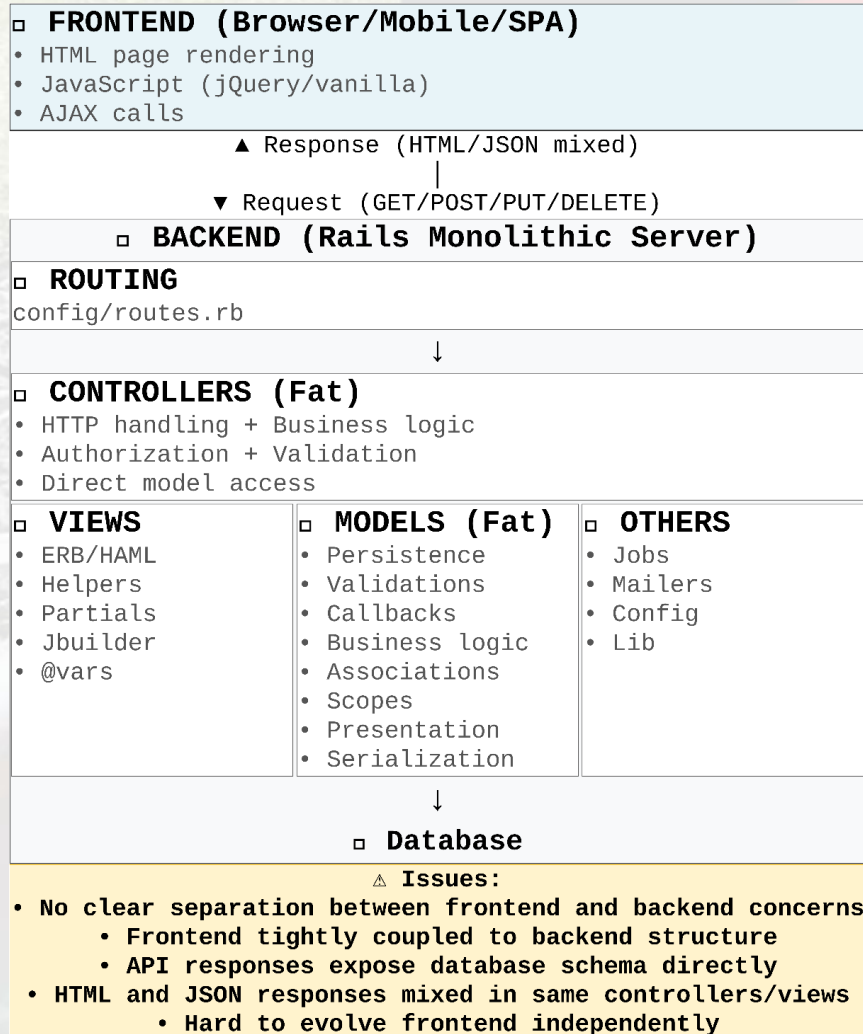
- **Convention over Configuration**
 - Draws inference on as much functionality as possible
 - Based on object names and project structure
 - Focus less on setups and more on product development
 - Grammatical Inflections on the number(singular/plural) and the case(e.g. I-my-me)
 - e.g. ***User*** model corresponds to ***users*** table of the database
- **3 layers: MVC**
 - **Models** (ActiveRecord)
 - **Views** (ERB HTML templates)
 - **Controllers** (Dataflow control)

2. Philosophy of "The Rails Way" -MVC vs. Layered-

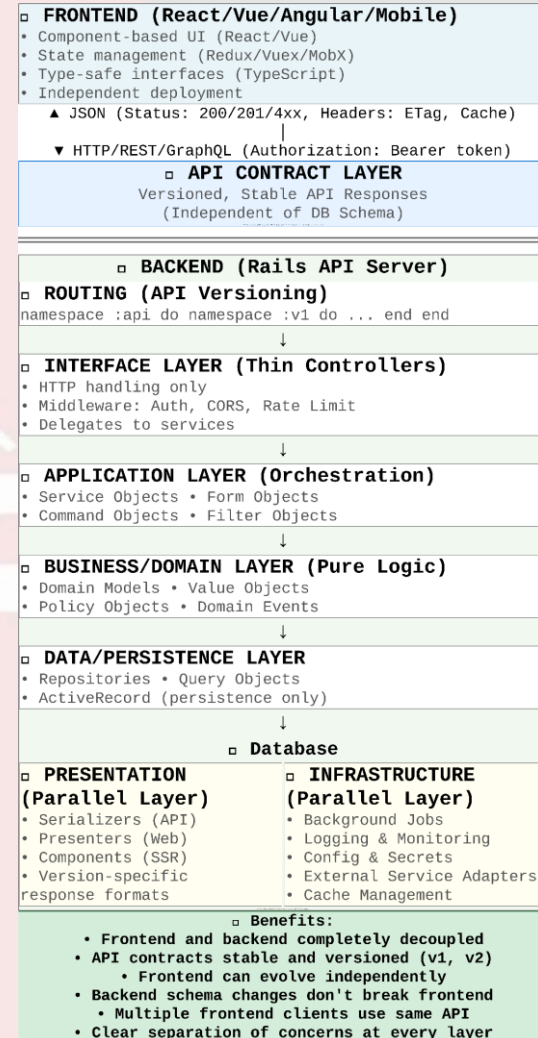
Extended/Layered Architecture

- **Structured abstractions: Role Segregations**
 - Encapsulated Logic
 - Focus less on the detailed logics and more on external behaviours
 - → Object Oriented
 - One-on-one single and clear responsibility assignments
 - Thinner controllers and models
- **5 (or more) specialized layers:**
 - Presentation
 - Business/Domain
 - Application/Service
 - Data/Persistence
 - Infrastructure

2. Philosophy of "The Rails Way" -MVC vs. Layered-



VS.



3. Key Responsibilities -MVC vs. Layered-



3. Key Responsibilities -MVC vs. Layered-

Responsibility	MVC	Layered
Presentation	• <i>app/controllers/*_controller.rb</i>: Instance variables • <i>app/helper/*_helper.rb</i>: Custom helper methods • ActiveModel::Serializers::JSON#as_json	• <i>app/serializers/*_serializer.rb</i> • <i>app/view/components/</i> • <i>app/view/presenters/</i>
Form Handling	• <i>app/controllers/*_controller.rb</i>: Strong parameters • <i>app/model/*.rb</i>: Validations	<i>app/forms/*_form.rb</i> (context-sensitive)
Business Logic	Scattered Everywhere • <i>app/controllers/*_controller.rb</i>(including concerns) • <i>app/model/*.rb</i>(including concerns) • <i>app/views/*.html.erb</i>(⚠ anti-pattern) etc.	• <i>app/model/*.rb</i> • <i>app/services/*_service.rb</i>
Authorization	• <i>app/controllers/*_controller.rb</i>: Callbacks • <i>app/model/*.rb</i>: Attributes	<i>app/policies/*_policy.rb</i> (single source of truth)
Data Access	Executed Everywhere • <i>app/controllers/*_controller.rb</i>(including concerns) • <i>app/model/*.rb</i>(including concerns) • <i>app/views/*.html.erb</i>(⚠ anti-pattern) etc.	• <i>app/commands/*_command.rb</i> • <i>app/repositories/*_repository.rb</i> • <i>app/queries/*_query.rb</i>

4. Key Components of The Extended Rails Way(Layered)



4. Key Components of The Extended Rails Way(Layered)

4-1. Presentation Layer

- [View Components](#): Explicit interfaces
- [Presenters](#): Context-aware formatting
- [Serializers](#): API contracts, versioning



4. Key Components of The Extended Rails Way(Layered)

4-2. Application/Service Layer

- [Service Objects](#): Single business operation
- [Form Objects](#): Multi-model operations, context validations
- [Filter Objects](#): User-driven queries



4. Key Components of The Extended Rails Way(Layered)

4-3. Business/Domain Layer

- [Domain Models](#): Rich behavior, no persistence details
- [Policy Objects](#): Authorization rules
- [Value Objects](#): Immutable domain concepts



4. Key Components of The Extended Rails Way(Layered)

4-4. Data/Persistence Layer

- [Repositories](#): Data access abstraction
- [Query Objects](#): Reusable complex queries
- [ActiveRecord](#): Stripped down to persistence only



5. Pros & Cons

The logo for RAILS, featuring a stylized white train engine icon on the left and the word "RAILS" in a bold, white, sans-serif font to its right. The entire logo is centered within a large, semi-transparent red circle that is itself centered on a grey rectangular background. This background is superimposed over a blurred photograph of a railway track receding into the distance under a bright sky.

RAILS

5. Pros & Cons

The Traditional MVC Architecture

- 👍 **Rapid Development:** Accessible built-in commands and generators
- 👍 **Convention over Configuration:** More productive with the least extensive configurations
- 👍 **Rich Ecosystem of Gems:** The maximum focus on [Core Subdomains](#)
- 👍 **Plenty of Resources:** Accessible to large supportive communities, documents, tutorials and other sources of information
- 👍 **Built-in Security Features:** Built-in security against web vulnerabilities like XSS, CSRF, and SQL injection
- 👍 **Best Practice Provisions:** Maintainable and reliable codebases
- 👍 **Programmer Happiness:** *"Do what you love and success will follow."*

5. Pros & Cons

The Traditional MVC Architecture

- 👎 **Steep Learning Curve for Beginners:** Required to learn how "Magics" work
- 👎 **Less Flexibility at Scale:** Difficult to find the best "The **Extended** Rails Way"
- 👎 **Performance Concerns at Scale:** Not optimal choice for high concurrency or low-latency responses
- 👎 **Deployment Complexity:** Required to manage various components like asset pre-compilation, database migrations, and background job workers, requiring some DevOps knowledge.
- 👎 **Black boxed Underlying Processes:** Profound mechanism make it for developers harder to debug and customise the app

5. Pros & Cons

The Extended/Layered Architecture

- 👍 **Maintainability:** Clear separation, easy to locate code
- 👍 **Testability:** Isolated layers, less framework coupling
- 👍 **Scalability:** Linear growth, independent layers
- 👍 **Flexibility:** Swap implementations, context-sensitive behavior
- 👍 **Clarity:** Code speaks business language
- 👎 **Initial Complexity:** More files, upfront design, steeper learning curve
- 👎 **Slower Start:** No scaffolding support, overhead for simple features
- 👎 **Team Alignment:** Everyone must follow patterns consistently
- 👎 **Debugging:** Longer call stacks, more indirection
- 👎 **Migration Costs:** Continuous factoring effort for existing apps

6. Pragmatic Adoption Strategy



6. Pragmatic Adoption Strategy

✓ Appropriate Cases - Enough Return of Investment(ROI)

- Beyond MVP stage
- Teams of more than 3 developers
- Complex business domains
- Long-term maintenance
- Multiple API clients
- Performance concerns due to the traditional MVC architecture

6. Pragmatic Adoption Strategy

✗ Inappropriate Cases - Poor Return of Investment(ROI)

- Simple CRUD actions only
- Prototypes/MVPs
- 1-2 person teams
- Short-lived projects



6. Pragmatic Adoption Strategy

Best Practice

1. **Start with MVC:** The traditional architecture for new projects
2. **Keep your eyes peeled on pain thresholds:**
 - Multiple models with 200 lines
 - Multiple controllers with complex workflows
 - Slow and unmaintainable tests(including too many untestable behaviours)
3. **Extract gradually:** Refactor specific areas as needed
4. **Follow conventions:** Maintain Rails idioms in new layers(e.g. ActiveRecord)
5. **Keep hybrid:** Traditional MVC where simple, layered where complex

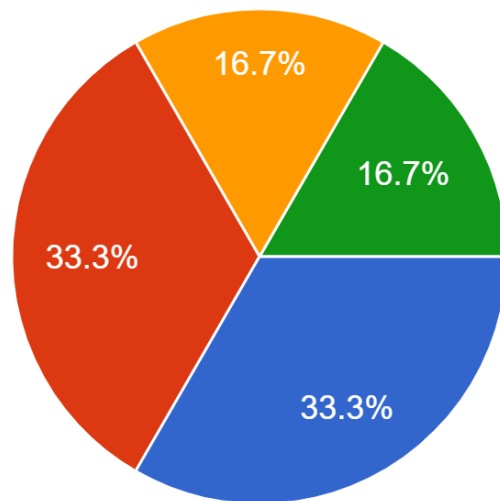
7. Survey and Answers



7. Survey and Answers

Have you ever failed to arrive at an agreement with your Rails app architecture with the keyword "The Rails Way"?

6 responses



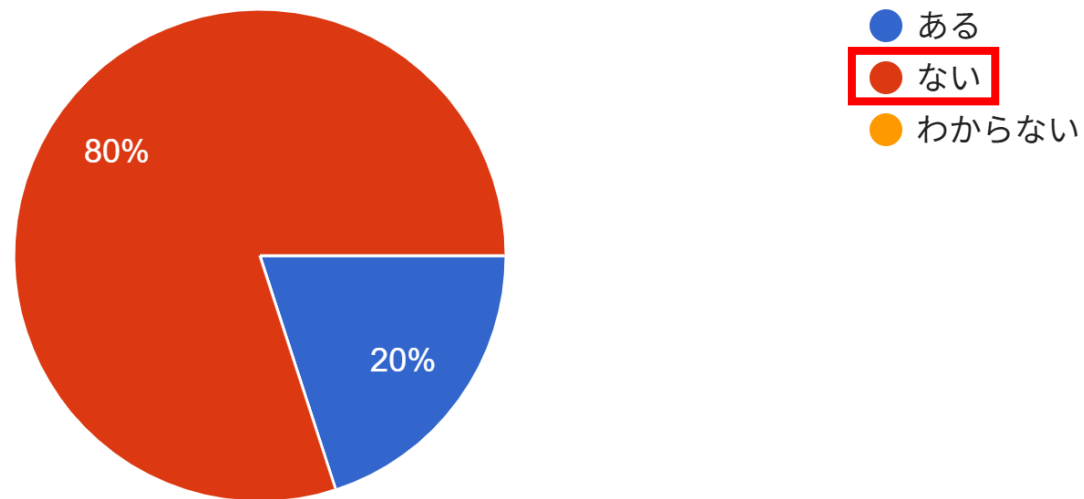
I don't know

I don't understand this question

7. Survey and Answers

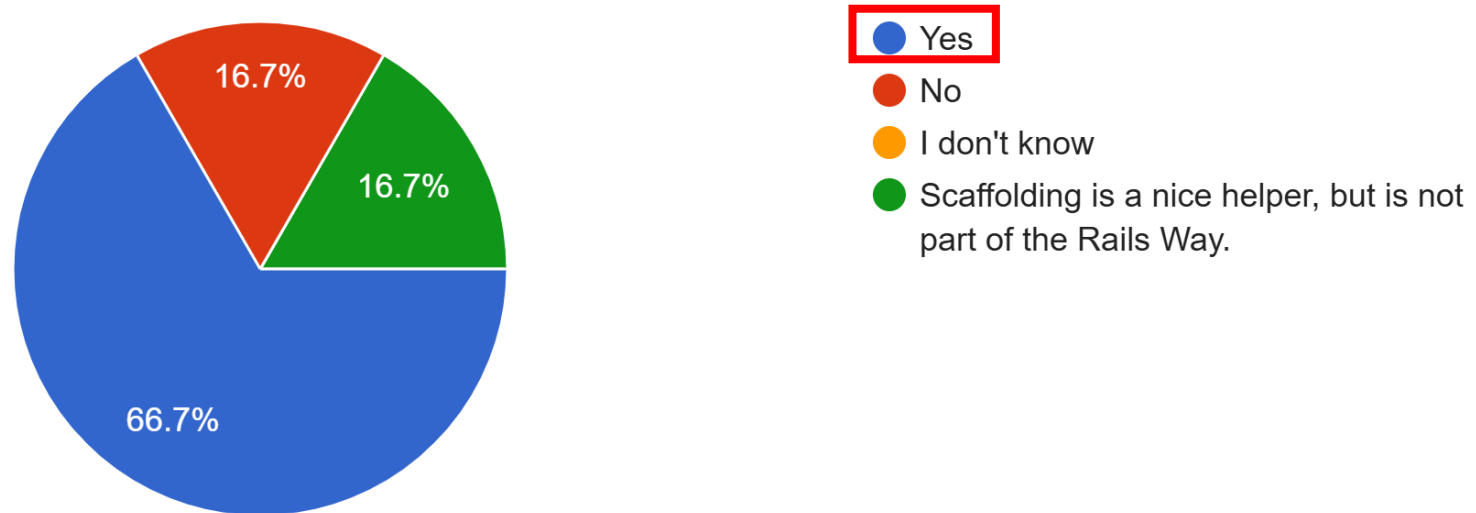
"The Rails Way" を合言葉にアーキテクチャーの合意形成に失敗した経験はありますか？

5 responses



7. Survey and Answers

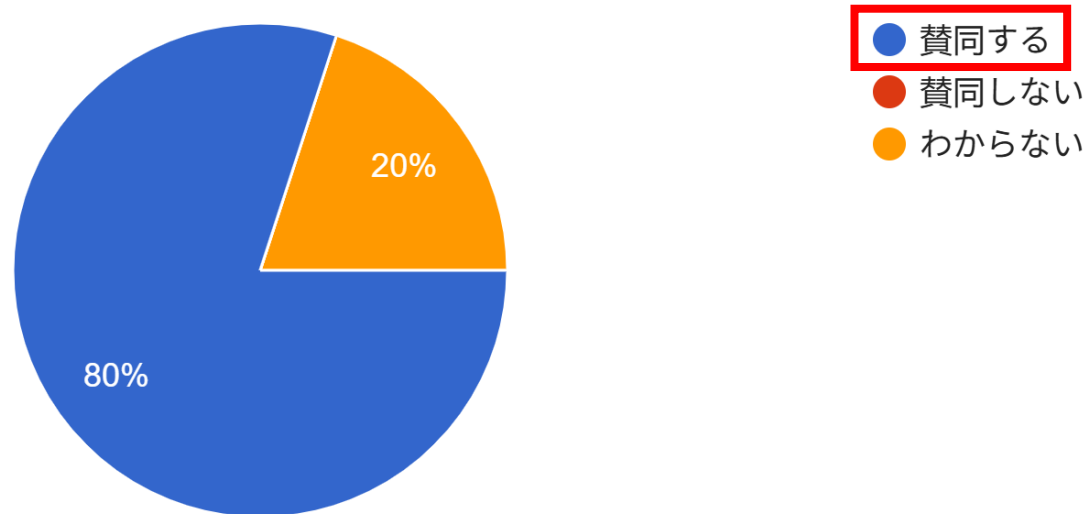
Here is the concept that the ecosystem generated by `bin/rails g scaffold ...` is treated as "The Rails Way in a narrow sense". Do you agree with it? * e.g.) MVC, CoC, Responsibility segregation
6 responses



7. Survey and Answers

Scaffold のエコシステムが「狭義の The Rails Way」という言説がありますが、賛同しますか？ ※
最低限の MVC と CoC

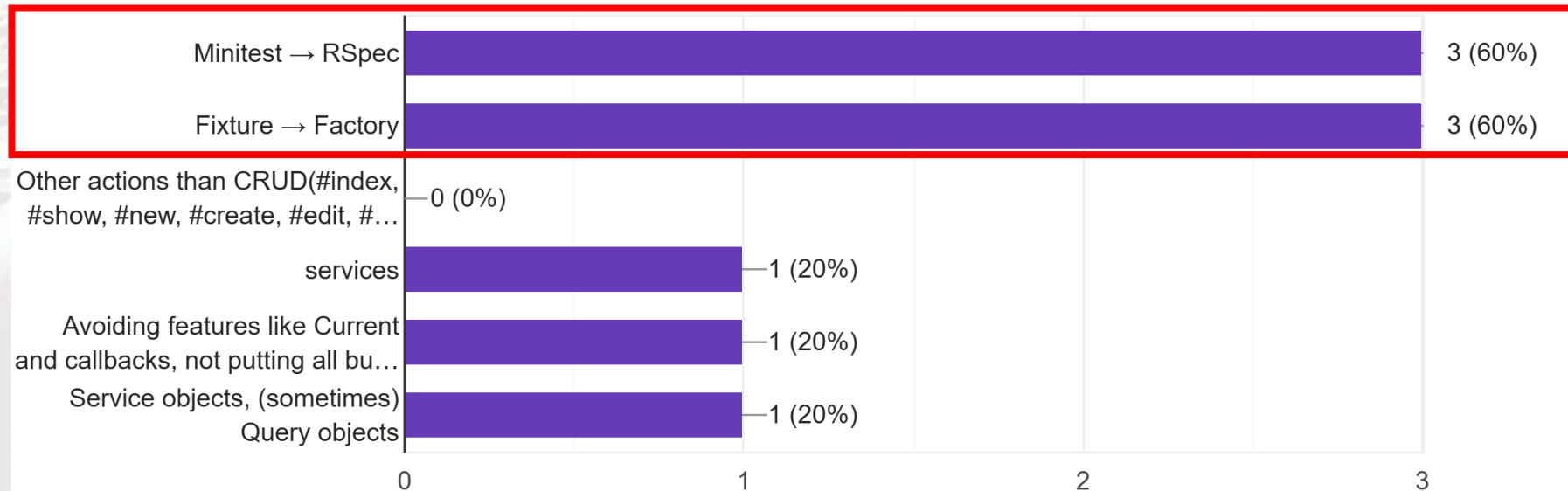
5 responses



7. Survey and Answers

Among the following components, what gets your Rails app out of "The Rails Way in a narrow sense"?

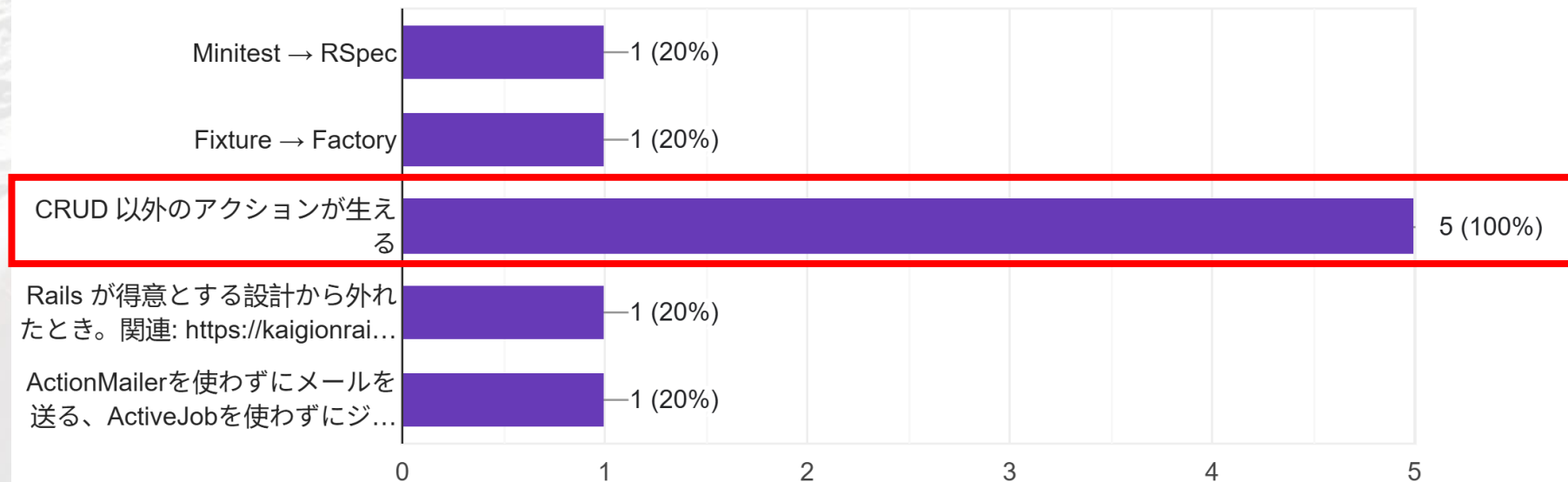
5 responses



7. Survey and Answers

「狭義の The Rails Way」の観点で、以下のエコシステムの中で置き換えられたら「それはもはや The Rails Way ではない」と考えるものを全て選んで下さい。

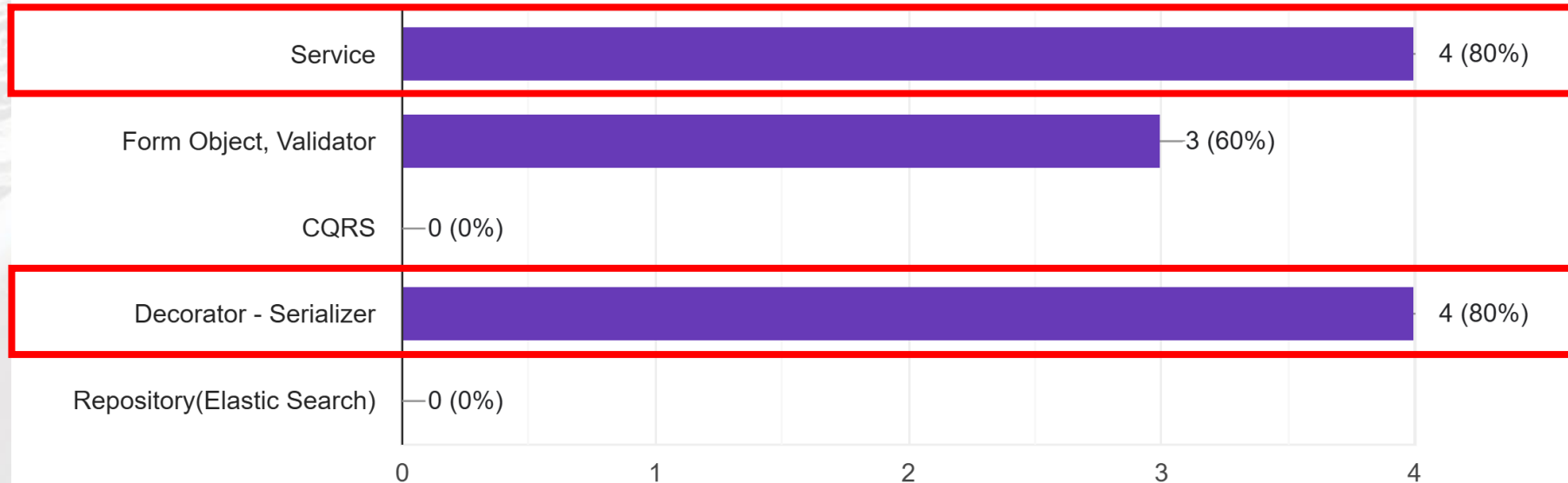
5 responses



7. Survey and Answers

What have you got in trouble with among the following ecosystem of "The Rails Way in a broader sense"?

5 responses

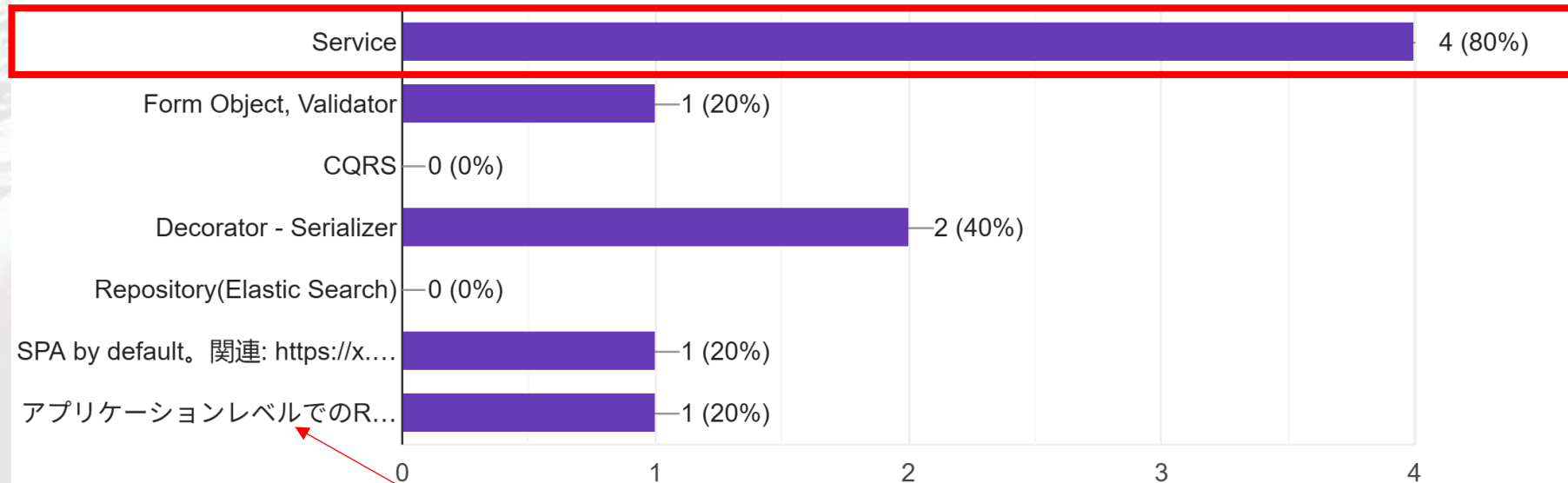


7. Survey and Answers

「広義の The Rails

Way」において、あなたが最も苦しめられたエコシステムを全て選んで下さい。

5 responses



Rails-like arbitrary method implementations based on CoC at the application layer(including multiple uses of `method\_missing`)

7. Survey and Answers

Q. Please feel free to describe the requirements which get your Rails app on "The Rails Way"? - International Rubyists

- Not packing everything under umbrella of "Services", not overusing dry-rb, accepting ActiveRecord and business logic can be coupled
- Following Rails' best practices, sticking to community favourites. Though personally I prefer to diverge from the Rails Way in multiple parts, I stick closer to it, when working with a growing team.
- Rails is like Lego, they give you the bricks, and you build your own spaceship.
- Sometime the decision on Rails are less optimized because it has to be abstracted.

7. Survey and Answers

Q. Please feel free to describe the requirements which get your Rails app on "The Rails Way"? - Japanese Rubyists(Partly extracted and processed)

- As long as everybody recognize what the architecture is, the Rails application is on "The Rails Way"
- rails new + scaffold => "The Rails Way"
- CoC + RESTful resource design + Concentrate business logics on Model layer => "The Rails Way"
- The practical ecosystem for the value maximization of delivery of the product is "The Rails Way" because **no value means no survival** for startups

8. Summary

The logo features a stylized white train engine icon on the left, composed of a semi-circle and a vertical line with small square details. To the right of the icon, the word "RAILS" is written in a bold, white, sans-serif font. The entire logo is centered within a large, semi-transparent red circle that is itself centered on a background image of railroad tracks receding into the distance under a bright sky.

RAILS

8. Summary

- **Traditional MVC:** Short path from idea to implementation, perfect for MVPs
 - → "The Rails Way" in a **narrower** sense
- **Layered Approach:** Clear boundaries, testable, maintainable at scale
 - → "The Rails Way" in a **broader** sense
- **Pragmatic Migration:** Start simple, add structure when complexity demands it.
- **Long-term Maintenance:** Keep refactoring guided by real pain points
- **Narrower "The Rails Way":** Most answers agreed with **CoC + MVC**
- **Broader "The Rails Way":** Most answers were unhappy with **Service Objects**
- **Out of "The Rails Way":** The case of **replacement of default ecosystem** among international Rubyists, in contrast to that of **definition of other actions than CRUD** among Japanese Rubyists

8. Summary

To cut a long story short...

"Think Less, Create More!" is The Rails Way!

9. References

The logo for RAILS, featuring a stylized white 'R' composed of several small squares, followed by the word 'AILS' in a bold, white, sans-serif font. The entire logo is centered within a large, semi-transparent red circle that is itself centered on a background image of railroad tracks receding into the distance under a bright sky.

RAILS

9. References

- Vladimir Dementyev, [*Layered Design for Ruby on Rails Application*](#), Birmingham, 2023
- Hayato Ishida, [*Layered Design for Ruby on Rails Application - Full Summary*](#), Tokyo, 2025
- Hayato Ishida, [*Layered Design for Ruby on Rails Application - Concise Summary*](#), Tokyo, 2025
- [*"The Rails Way" Survey - EN*](#)
- [*"The Rails Way" Survey - JP*](#)
- [*Statistics of "The Rails Way" Survey*](#)
- [*dry-rb Gems*](#)
 - Last Accessed At: 12 November 2025

